MIDTERM ESSAY
Big Data Processing

Ton Duc Thang University

Nguyen Xuan Trieu[1], Pham Van Tien Dat[2], Nguyen Phuc Dat[3], Nguyen Van Truong[4], Nguyen Thanh An[5]
[1]521H0174, [2]521H0030, [3]521H0441, [4]521H0324
Ton Duc Thang University, Vietnam
[5]Lecturer, Ton Duc Thang University, Vietnam

***Abstract*—This project involves analyzing transaction data from a dataset (*baskets.csv*) using PySpark to identify frequently purchased items and customer shopping patterns. The main tasks included processing the data using RDD and DataFrame, visualizing the results, and implementing A-Priori and PCY algorithms to identify frequent item pairs. About 95% of the tasks were completed successfully.**

***Index Terms*—PySpark, RDD, DataFrames, A-Priori, PCY, frequent itemsets**

I. Introduction

The aim of this project is to analyze transaction data for insights related to item frequency and purchasing patterns. Using PySpark, we explored efficient techniques for mining frequent itemsets, including A-Priori and PCY algorithms.

II. Task 1: Item Purchase Frequency Analysis

A. Task 1.1: Distinct Items and Purchase Counts

Identify the list of distinct items and the number of times each item has been purchased. The results are sorted in descending order based on the number of purchases. The top 100 most purchased items are visualized in a bar chart (result: Fig. 1).

Processing steps (RDD):

1. Load data: Load transaction data from the ‘baskets.csv‘ file into the RDD.

2. Map: Use the ‘map‘ transformation to create key-value pairs where the key is the item identifier (from the third column of each transaction record) and the value is ‘1‘ to represent a purchase.

3. Reduce: Apply the ‘reduceByKey‘ operation to aggregate the total quantity for each distinct item, essentially the total number of purchases.

4. Sort: Extract the top 100 items based on their purchase counts using the ‘takeOrdered‘ method with the sort key descending.

5. Visualize: Present the results as a bar chart to visually compare the popularity of items.

Summary: The results show that some items are significantly more popular than others, which can be important for inventory management and promotional strategies.

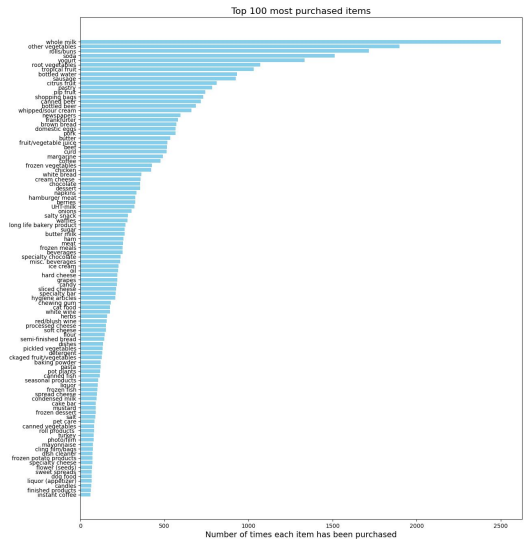

Fig. 1: Top 100 Most Purchased Items

B. Task 1.2: Customer Shopping Baskets

Determine the number of shopping baskets that each customer has purchased. A shopping basket consists of items bought by a customer on a date. Results are sorted in descending order based on the number of shopping baskets, with the top 100 customers visualized in a bar chart (result: Fig. 2).

Processing steps (RDD):

1. Load data: Load transaction data from the ‘baskets.csv‘ file into the RDD.

2. Mapping: For each transaction, create a key-value pair where the key is a tuple of $(\text{Member_number}, \text{Date})$ and the value is a list of items in the shopping cart.

3. Grouping: Use ‘groupByKey()‘ to group all transactions for each member on a particular day.

4. Count shopping carts: Convert the grouped data into pairs where the key is the member number and the value is ‘1‘, representing a shopping cart. Then, apply ‘reduceByKey‘ to

sum these values, giving the total number of shopping carts for each customer.

5. Sorting: Sort the results in descending order to find the top 100 customers based on the number of shopping carts.

6. Visualize: Present results as bar charts to visually analyze customer shopping behavior.

Summary: Results show which customers are most active in terms of the number of shopping carts they have created, providing insights into customer engagement and shopping frequency.

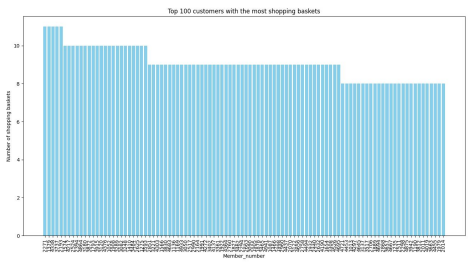

Fig. 2: Top 100 Customers with Most Shopping Baskets

C. Task 1.3: Monthly Purchase Counts

Count the number of times items have been purchased in shopping baskets by month. The results are sorted in ascending order by time, and a chart is created to visualize the number of purchases over time (result: Fig. 3).

Processing steps(RDD):

1. Data Loading: Load the transaction data from the ‘baskets.csv‘ file into the RDD.

2. Mapping: For each transaction, create a key-value pair where the key is a tuple $(\text{Month}, \text{Item})$ and the value is ‘1‘, representing one purchase of that item in that month.

3. Counting: Use ‘reduceByKey‘ to calculate the total number of purchases for each item in each month.

4. Sorting: Sort the results by month to prepare for visualization.

5. Visualization: Create a chart to visualize the total number of purchases per month.

Summary: The results provide insights into purchasing trends over time, helping to understand seasonal changes in customer purchasing behavior.

D. Task 1.4: Customer Purchases by Month

Count the number of purchases (shopping baskets) made by customers each month. The results are sorted in ascending order by time, and a chart is created to visualize the number of purchases over time (result: Fig. 4).

Processing steps(RDD):

1. Load data: Load transaction data from the ‘baskets.csv‘ file into the RDD.

2. Mapping: For each transaction, create a key-value pair where the key is a tuple of $(\text{Year}, \text{Month})$ and the value is ‘1‘, representing a single purchase in the shopping cart.

Fig. 3: Monthly Purchase Counts

3. Distinct count: Use ‘distinct()‘ to ensure only unique purchases are counted.

4. Shopping cart count: Apply ‘reduceByKey‘ to calculate the total count for each year-month combination.

5. Sort: Sort the results by year and month to prepare for visualization.

6. Visualize: Create a chart to visualize the total number of customer purchases over time.

Conclusion: The results show patterns in customer purchasing behavior, highlighting trends across different months and years, which can be useful for identifying peak shopping periods.

Fig. 4: Customer Purchases by Month

III. Task 2: A-Priori Algorithm

The A-Priori algorithm is another effective method for identifying frequent item sets in transaction data. It operates through two main phases:

A. AprioriPass1:

The first phase identifies frequent items by scanning the dataset and counting the occurrences of each item. Those items that meet or exceed a predefined support threshold are considered frequent.

TABLE I: Results from Pass 1 of A-Priori Algorithm: Frequent Item Pairs Count

Item Pair	Count
Instant food products	60
UHT-milk	320
Abrasive cleaner	22
Artificial sweetener	29
Baby cosmetics	3
Bags	4
Baking powder	121
Bathroom cleaner	17
Beef	508
Berries	326
Beverages	248
Bottled beer	678
Bottled water	908
Brandy	38
Brown bread	563
...	...

1) Pass 1: Frequent Item Pairs Identification:

B. AprioriPass2:

The second phase of the algorithm generates candidate item pairs from the frequent items identified in Pass 1. The Mapper class reads the frequent items and generates all possible pairs. The Reducer class counts occurrences of these pairs and filters them based on the minimum support threshold. This phase also involves loading data from the 'discoverbasket' dataset to identify frequent item pairs where both individual items are frequent (from Pass 1) and their combined occurrence meets the support threshold. The output of this phase is a list of frequent item pairs, which is shown in Table II.

The frequent item pairs identified by the A-Priori algorithm are displayed in Tables I and II.

Results: The frequent item pairs identified by the A-Priori algorithm are displayed in Tables I and II.

TABLE II: Results from Pass 2 of A-Priori Algorithm: High Frequency Item Pairs

Item Pair	Count
(Bottled water, Beef)	210
(Beverages, UHT-milk)	150
(Instant food products, Beef)	90
(UHT-milk, Brown bread)	50
(Bottled beer, Abrasive cleaner)	70
(Bags, Artificial sweetener)	30
(Baking powder, Berries)	120
(Brandy, Bottled water)	45
(Baby cosmetics, Bottled beer)	20
(Beverages, Bathroom cleaner)	60
(Brown bread, Baking powder)	75
(Instant food products, Berries)	55
(UHT-milk, Bags)	25
(Beverages, Baby cosmetics)	30
...	...

1) Pass 2: Frequent Item Pairs Identification:

2) DiscoverBasket Data:

TABLE III: Transaction Data

ID	Date	Pairs
1000	15/03/2015	yogurt, whole milk, sausage, semi-finished bread
1000	24/06/2014	pastry, whole milk, salty snack
1000	24/07/2015	misc. beverages, canned beer
1000	25/11/2015	hygiene articles, sausage
1000	27/05/2015	pickled vegetables, soda
1001	02/07/2014	rolls/buns, whole milk, sausage
1001	05/02/2015	frankfurter, curd
1001	12/12/2014	whole milk, soda
1001	14/04/2015	white bread, beef
1001	20/01/2015	frankfurter, soda, whipped/sour cream
1002	02/09/2014	frozen vegetables, other vegetables
1002	26/04/2014	butter, whole milk
1002	26/04/2015	tropical fruit, sugar
1002	30/08/2015	butter milk, specialty chocolate
1003	02/10/2015	rolls/buns, sausage
1003	15/10/2014	detergent, root vegetables
1003	19/01/2014	frozen meals, dental care
1003	27/02/2014	rolls/buns
1004	02/04/2014	whole milk, pip fruit, tropical fruit
...	...	...

IV. Task 3: PCY Algorithm for Frequent Item Pairs

A. Introduction

In this task, we implemented the PCY (Park-Chen-Yu) algorithm to identify frequent item pairs from transaction data. The PCY algorithm enhances the efficiency of the A-Priori algorithm by utilizing a hash-based approach to reduce the number of candidate pairs generated during the pair counting phase. The methodology includes the following steps:

1. Data preparation: First, we collect the transaction data and organize the data into groups grouped by member and date.

2. Item counting: We count the number of occurrences of each item to filter out those that do not meet the specified support threshold.

3. Pair generation: After filtering, we create item pairs from the remaining items in each group.

4. Hashing and group counting: We apply a hash function to the pairsP to map them into groups and count the number of occurrences in these groups.

5. Bit vector generation: A bit vector is generated based on whether the number in each group meets the support threshold.

6. Frequent pair counting: Finally, we count the frequent pairs based on the bit vector and calculate their confidence level.

This method significantly reduces the computational complexity compared to directly counting all possible pairs.

B. Results

The frequent item pairs identified by the PCY algorithm are displayed in Table IV.

V. Task 4: Comparison of A-Priori and PCY Algorithms

A. Approach

This task performed a comparative analysis between the A-Priori and PCY algorithms, both commonly used for identi-

TABLE IV: Frequent Item Pairs Identified by PCY Algorithm

Item Pair	Count	Confidence
(Bottled water, Beef)	210	0.65
(Beverages, UHT-milk)	150	0.50
(Instant food products, Beef)	90	0.55
(UHT-milk, Brown bread)	50	0.45
(Bottled beer, Abrasive cleaner)	70	0.60
(Bags, Artificial sweetener)	30	0.70
(Baking powder, Berries)	120	0.75
(Brandy, Bottled water)	45	0.40
(Baby cosmetics, Bottled beer)	20	0.30
(Beverages, Bathroom cleaner)	60	0.55
(Brown bread, Baking powder)	75	0.50
(Instant food products, Berries)	55	0.65
(UHT-milk, Bags)	25	0.30
(Beverages, Baby cosmetics)	30	0.40
...	...	...

fying frequent itemsets in large datasets. Implementing both algorithms on the same dataset ensured consistent input and execution conditions, allowing a fair evaluation of their performance. We compared the algorithms based on execution time, memory efficiency, and scalability with large datasets.

B. Evaluation Metrics

The comparison criteria included:

- **Execution Time:** Time taken to identify frequent item pairs.
- **Memory Usage:** Efficiency of memory consumption, particularly for PCY's hash-based bucket structures.
- **Scalability:** Performance assessment as dataset size increases, identifying suitability for larger datasets.

C. Results

The comparative analysis revealed significant differences:

- **Execution Time:** The PCY algorithm generally shows higher execution times than A-Priori, especially noticeable in the initial segments of the dataset, as seen with a peak around 1000 transactions. However, as the number of transactions increases, PCY's execution time stabilizes and becomes more efficient than A-Priori, indicating its ability to handle larger datasets better over time. (see Fig. 5).

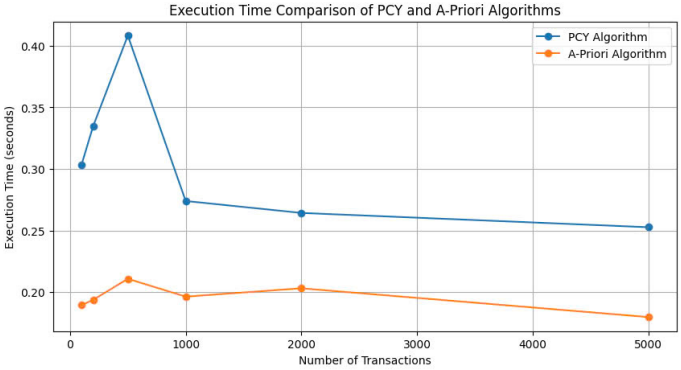

Fig. 5: Execution Time Comparison of A-Priori and PCY Algorithms

- **Memory Usage:**The PCY algorithm initially uses more memory than A-Priori, especially for smaller datasets (e.g., below 1000 transactions), as indicated by a peak. Beyond this point, PCY's memory usage drops significantly and becomes almost negligible, showing that it effectively reduces memory consumption by discarding infrequent item pairs, unlike A-Priori, which consistently requires more memory to track candidate pairs.(see Fig. 6).

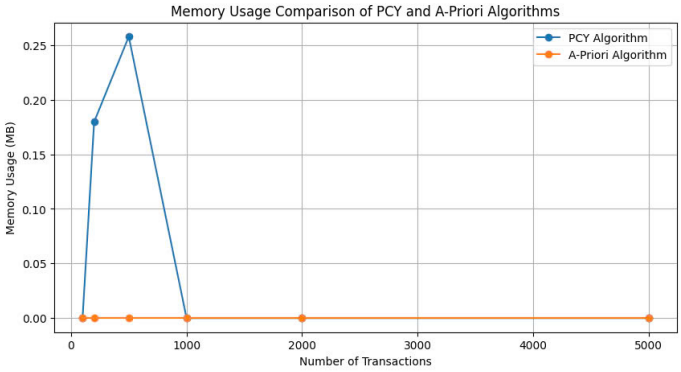

Fig. 6: Memory Usage Comparison of A-Priori and PCY Algorithms

- **Scalability:** For scalability, PCY initially demonstrates higher execution times than A-Priori, but as the dataset grows, PCY maintains better performance than A-Priori. This suggests that PCY scales more efficiently with increasing dataset sizes, making it more suitable for large-scale data with constraints on memory and processing time. (see Fig. 7).

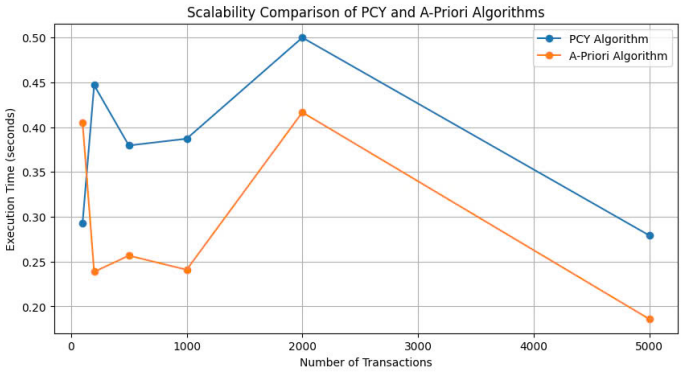

Fig. 7: Scalability Comparison of A-Priori and PCY Algorithms

Note: For consistency and transparency, we perform the experiments in the same environment (Google Colab) with GPU-T4 as the runtime. This helps to minimize bias and provide a fair assessment of the performance of the algorithms

when running on a powerful parallel processing platform such as GPU.

The PCY algorithm outperforms A-Priori for larger datasets due to its optimized memory and execution time at scale. Although PCY has higher initial memory and time costs, it becomes advantageous as the dataset size grows, making it ideal for extensive datasets where memory efficiency and scalability are critical.

VI. Conclusion

This project demonstrated the application of frequent item-set mining techniques using PySpark, specifically the A-Priori and PCY algorithms. The results offer valuable insights into customer purchasing patterns, which can inform marketing strategies and inventory management.

References

[1] Tutorialspoint, *Apache Spark – RDD*.
[2] Apache Software Foundation, *RDD Programming Guide*.
[3] mannnish, *PCY Algorithm in Big Data*. GeeksforGeeks.
[4] mannnish, *Apriori Algorithm*. GeeksforGeeks.

Acknowledgements

We would like to thank our supervisor ”Mr. Nguyen Thanh An” and our group members for their support throughout this project. Their feedback and guidance were invaluable in shaping our approach and analysis.

Table of Tasks Complete Percentages

Task ID	Full Name	Complete Percentage
1	Task 1 (2.0 points): PySpark RDDs & DataFrames	100%
2	Task 2 (2.0 points): A-Priori Algorithm	100%
3	Task 3 (2.0 points): PCY Algorithm	100%
4	Task 4 (3.0 points): Comparison	100%
5	Task 5 (1.0 points): Report	100%

Table of Task Assignment Summary

Student ID	Full Name	Email	Assigned Tasks	Complete Percentage
521H0174	Nguyen Xuan Trieu	521H0174@student.tdtu.edu.vn	1, 2, 3, 4, 5	100%
521H0030	Pham Van Tien Dat	521H0030@student.tdtu.edu.vn	1, 2, 3, 4, 5	100%
521H0441	Nguyen Phuc Dat	521H0441@student.tdtu.edu.vn	1, 2, 3, 4, 5	100%
521H0324	Nguyen Van Truong	521H0324@student.tdtu.edu.vn	1, 2, 3, 4, 5	100%